



ESCUELA DE
INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

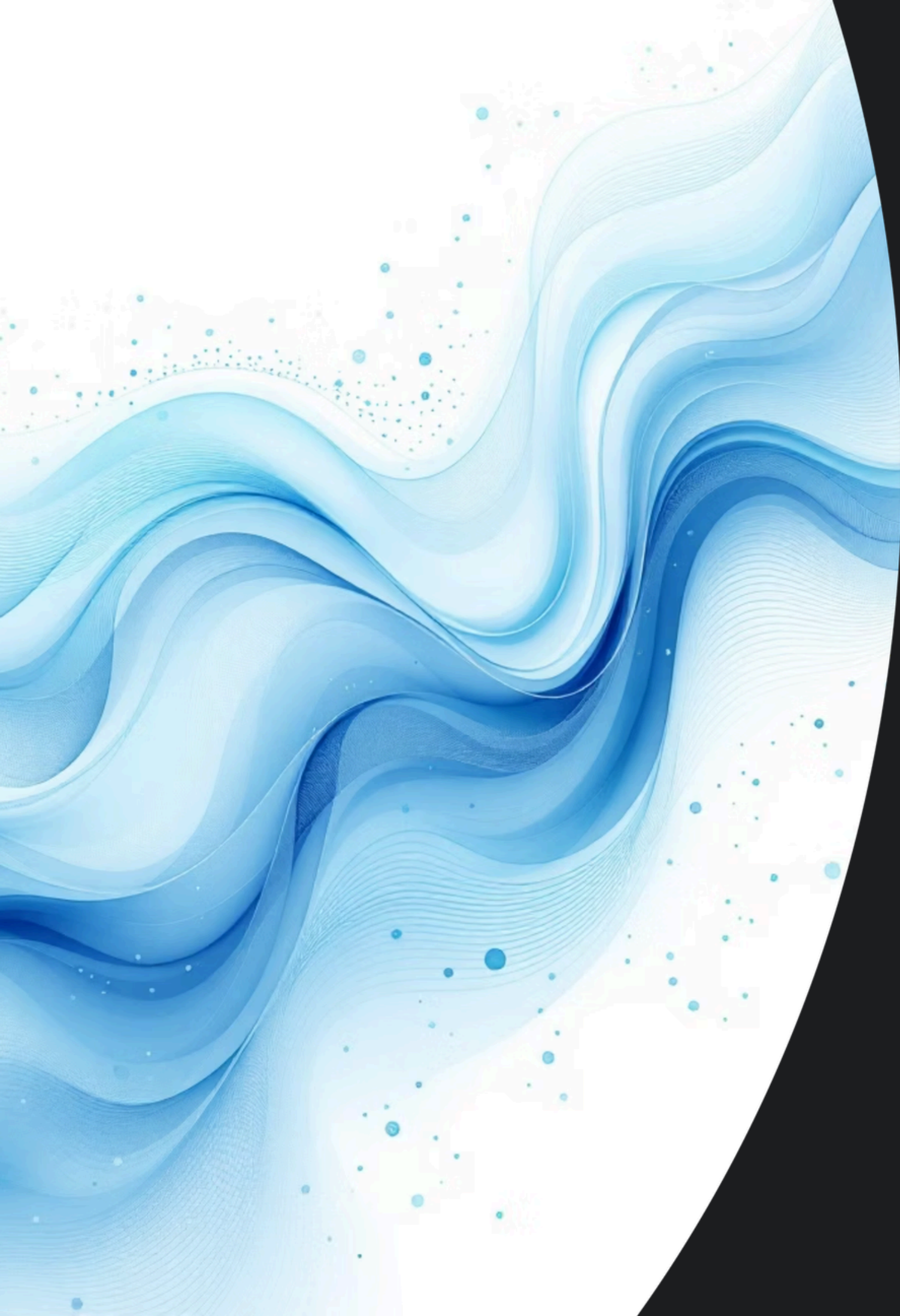


Día, Fecha:	Martes, 07 de abril
Hora de inicio:	12:20 p.m.

Programación de computadoras 2 P

Josué Alejandro Pérez Benito





Relaciones y Modelado en Bases de Datos Relacionales

Una guía técnica y pedagógica para comprender cómo se estructuran, vinculan e implementan los datos en MySQL — desde los conceptos fundamentales hasta el código SQL real.

ARQUITECTURA DE BASES DE DATOS

MYSQL · UML

Conceptos Fundamentales

Una base de datos relacional organiza la información en **tablas**, donde cada tabla representa una entidad del mundo real.



Tabla

Contenedor principal que agrupa datos de una misma entidad.
Ejemplo: clientes, productos.



Fila (Registro)

Representa una instancia individual de la entidad. Cada fila es única dentro de la tabla.



Columna (Atributo)

Define una propiedad específica de la entidad. Tiene un tipo de dato fijo: INT, VARCHAR, DATE.



Identificadores Únicos: Primary Key (PK)

¿Qué es una Llave Primaria?

La **PK** identifica de forma **única e irrepetible** cada fila de una tabla. MySQL garantiza su unicidad mediante un índice automático.

Reglas de Oro

- Nunca puede ser NULL — la fila dejaría de ser identificable.
- Debe ser **inmutable** — cambiarla rompe las referencias de otras tablas.
- Preferir valores **surrogados** (AUTO_INCREMENT) sobre datos naturales como el RFC.

Simple

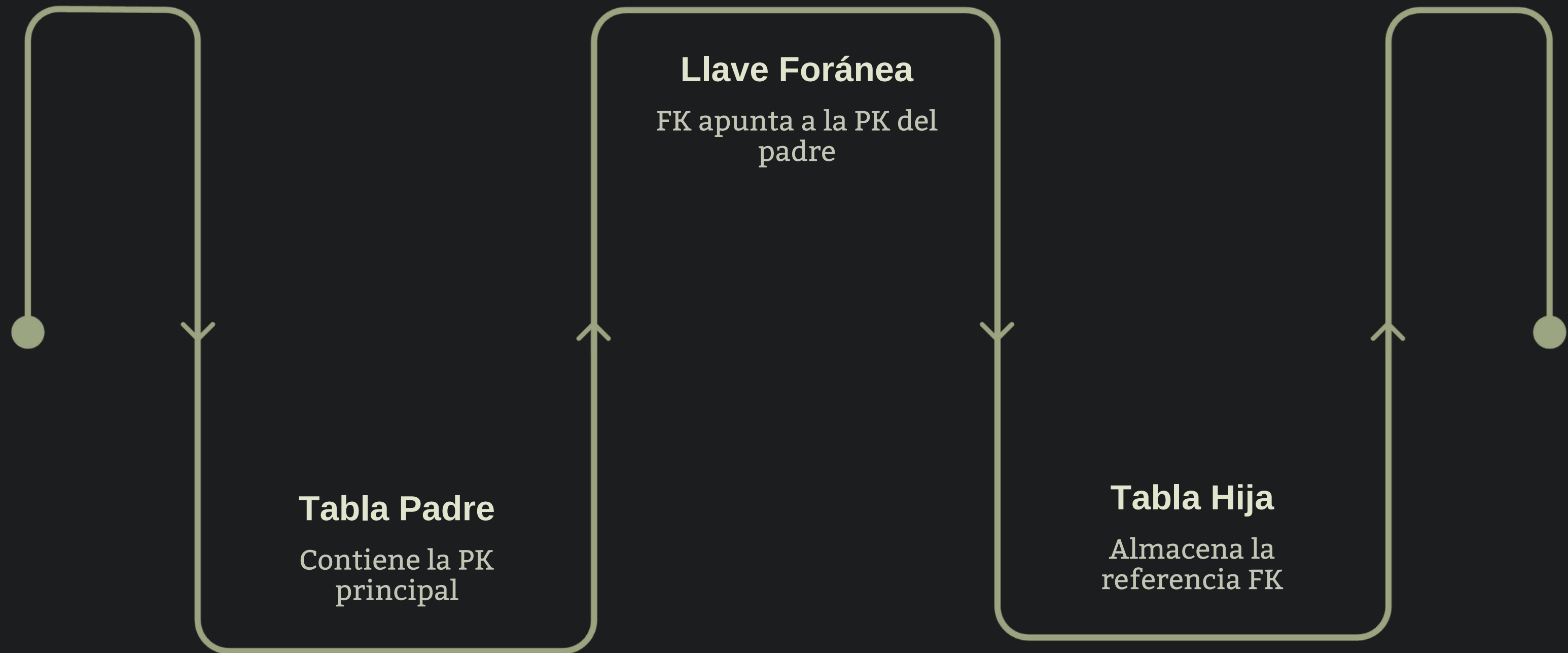
Un solo campo: `id_cliente INT
AUTO_INCREMENT`

Compuesta

Combinación de columnas: `PRIMARY KEY
(id_curso, id_alumno)`

Vínculos entre Tablas: Foreign Key (FK)

La **Llave Foránea** es el mecanismo que crea una relación formal entre dos tablas, garantizando que los datos referenciados existan.



En MySQL, la FK se declara con `FOREIGN KEY (col) REFERENCES tabla_padre(pk)`. Si el valor no existe en la tabla padre, la operación es **rechazada automáticamente** por el motor InnoDB, preservando la integridad referencial.

Tipos de Cardinalidad

1

Uno a Uno (1:1)

Una fila de A se relaciona con **exactamente una** fila de B.
Ejemplo: usuario → perfil_detalle. Se usa para separar datos sensibles o de acceso poco frecuente.

2

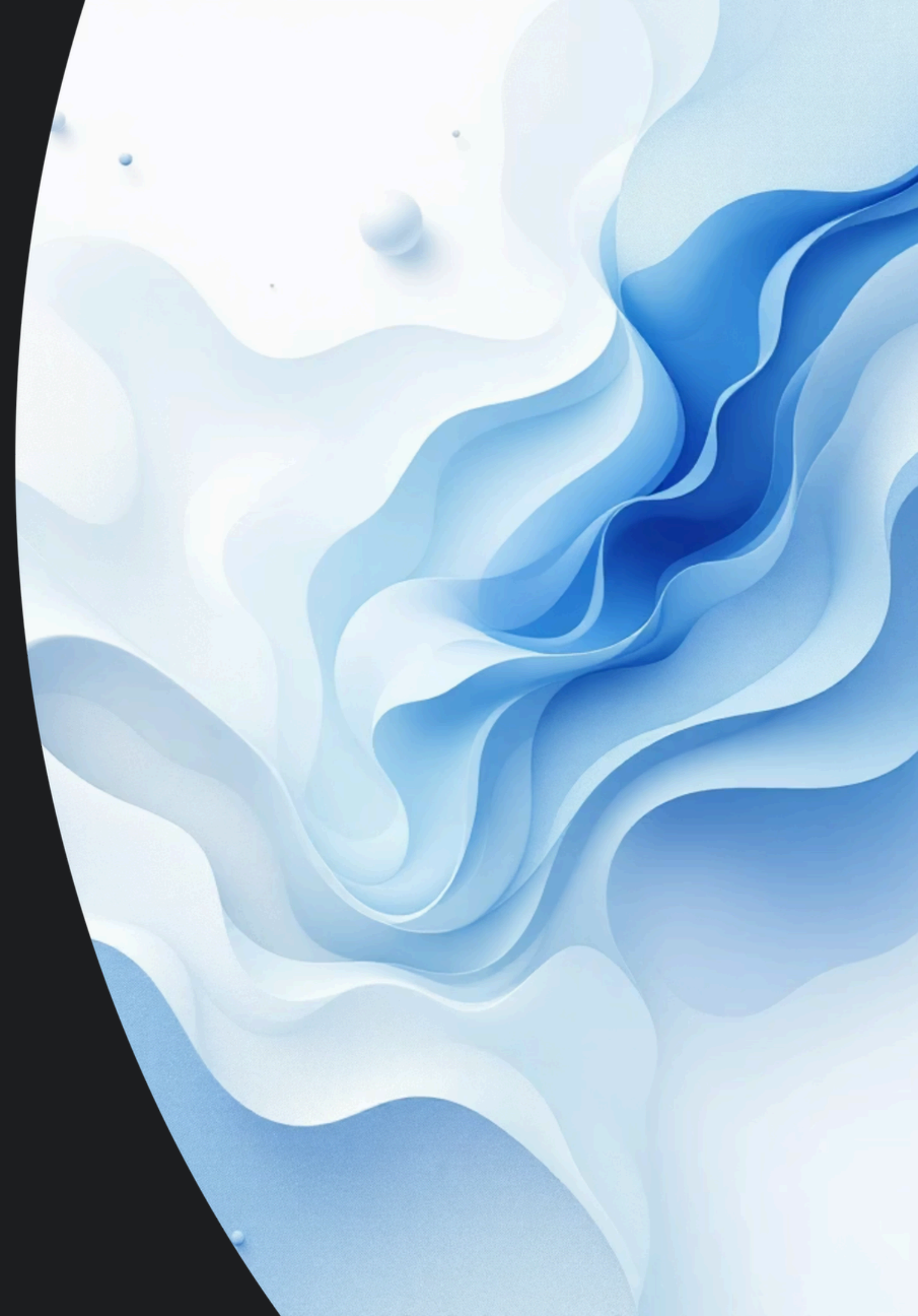
Uno a Muchos (1:N)

Una fila de A puede relacionarse con **múltiples** filas de B. Es la más común. Ejemplo: un cliente tiene muchos pedidos. La FK vive en el lado "muchos".

3

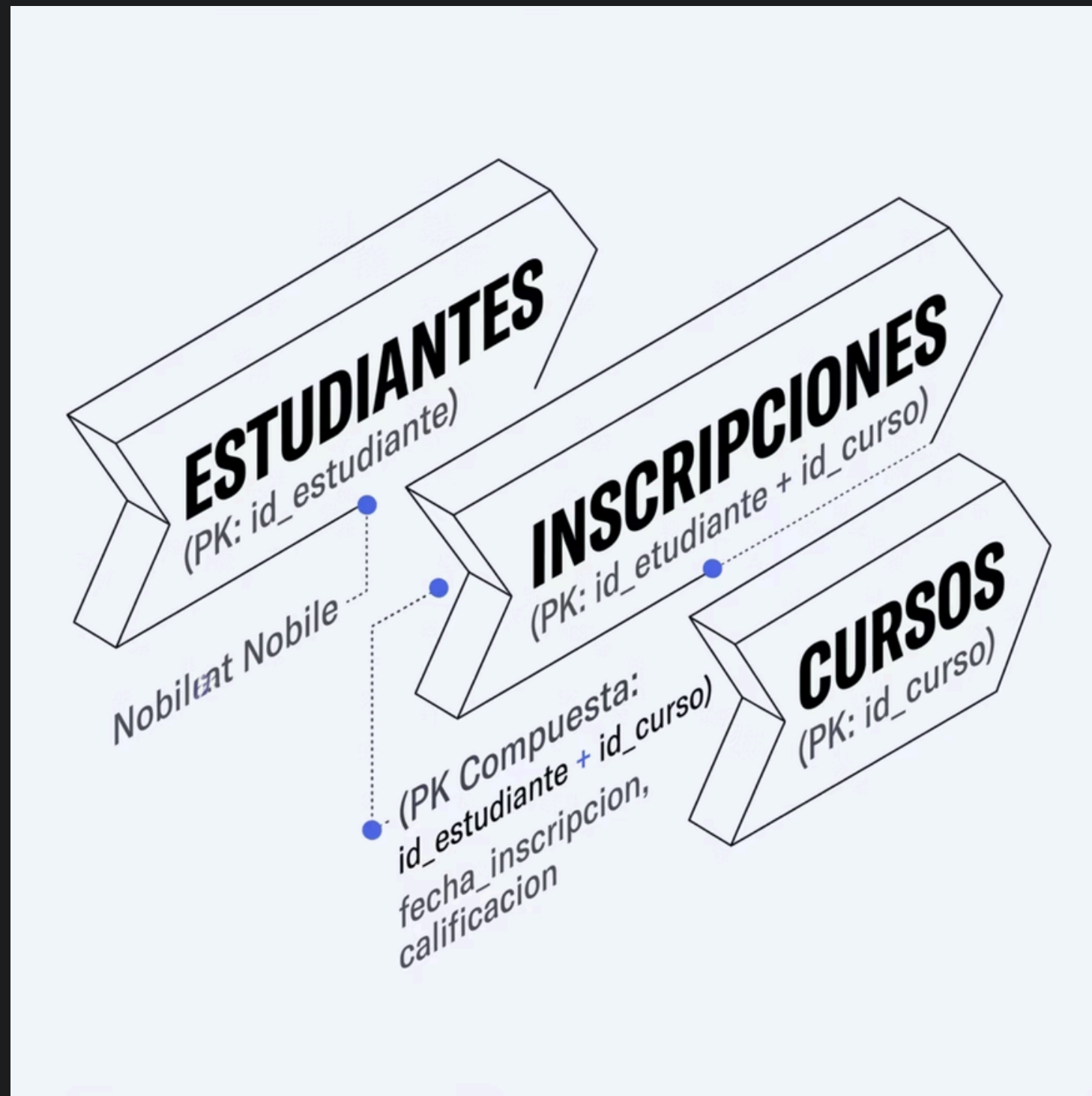
Muchos a Muchos (N:M)

Múltiples filas de A se relacionan con múltiples filas de B.
Ejemplo: estudiantes y cursos. Requiere una **tabla asociativa** para su resolución.



Resolución de Relaciones N:M

Las relaciones Muchos a Muchos no pueden implementarse directamente en SQL. Se resuelven mediante una **tabla asociativa (o de unión)** que descompone la relación en dos vínculos 1:N.



Beneficios de la Tabla Asociativa

- 1** Rompe la complejidad N:M en dos relaciones 1:N manejables.
- 2** Permite agregar **atributos propios** a la relación (ej. **fecha_inscripcion**, **calificacion**).
- 3** Su PK compuesta garantiza que no existan inscripciones duplicadas.

Modelado con UML: Tablas como Clases

En UML, cada tabla se representa como una **clase** dividida en tres compartimentos. Esta notación es el estándar para diseñar el esquema antes de escribir SQL.

Anatomía de una Clase UML

01

Nombre de la clase — corresponde al nombre de la tabla (Cliente).

02

Atributos con tipo de dato — cada columna con su tipo MySQL:
+id_cliente: INT.

03

Visibilidad — + público (columna accesible), - privado, # protegido.
Las PK se marcan con «PK».

Ejemplo: Clase Pedido

«Table»

Pedido

«PK» +id_pedido: INT

«FK» +id_cliente: INT

+fecha: DATE

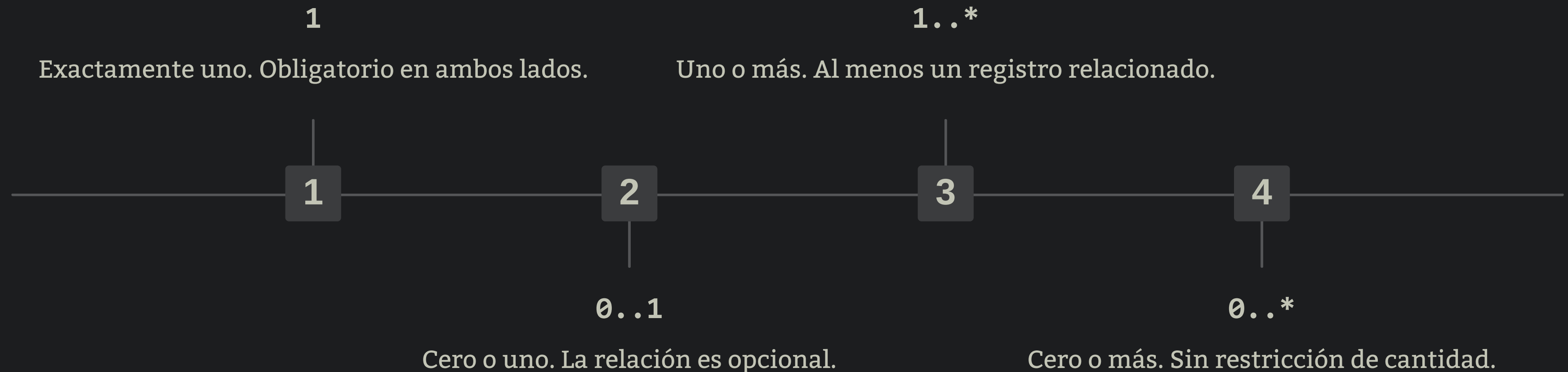
+total: DECIMAL(10,2)

+estatus: VARCHAR(20)

Los estereotipos «PK» y «FK» identifican visualmente el rol de cada atributo sin ambigüedad.

Relaciones en UML: Multiplicidades

Las líneas de asociación conectan clases y las **multiplicidades** definen cuántas instancias participan en cada extremo de la relación.



- ❏ La notación UML con multiplicidades es preferida sobre la "pata de gallo" en diseño formal, ya que es más precisa, legible en herramientas CASE y compatible con generación automática de código SQL.

Integridad Referencial

MySQL InnoDB permite definir **reglas de comportamiento** ante operaciones de borrado o actualización en la tabla padre mediante las cláusulas ON DELETE y ON UPDATE.

CASCADE

Propaga el cambio automáticamente a las filas hijas. Útil para borrado en cascada de datos dependientes.

SET NULL

Pone la FK en NULL en los registros hijos. La columna FK debe permitir nulos.

NO ACTION / RESTRICT

Rechaza la operación si existen registros hijos. Es el comportamiento **más seguro por defecto**.

SET DEFAULT

Asigna un valor por defecto a la FK. Poco soportado en MySQL InnoDB; usar con precaución.

De la Teoría a la Implementación SQL

Así se traduce un diagrama UML con relación **1:N** entre Cliente y Pedido a código MySQL real:

```
-- Tabla Padre
CREATE TABLE clientes (
  id_cliente INT AUTO_INCREMENT,
  nombre      VARCHAR(100) NOT NULL,
  email       VARCHAR(150) UNIQUE,
  PRIMARY KEY (id_cliente)
) ENGINE=InnoDB;
```

```
-- Tabla Hija con FK
CREATE TABLE pedidos (
  id_pedido  INT AUTO_INCREMENT,
  id_cliente INT NOT NULL,
  fecha      DATE NOT NULL,
  total      DECIMAL(10,2),
  PRIMARY KEY (id_pedido),
  FOREIGN KEY (id_cliente)
    REFERENCES clientes(id_cliente)
    ON DELETE RESTRICT
    ON UPDATE CASCADE
) ENGINE=InnoDB;
```

Puntos Clave del DDL

- **ENGINE=InnoDB** es obligatorio para que MySQL aplique la integridad referencial.
- La PK de clientes es referenciada directamente por la FK de pedidos.
- **ON DELETE RESTRICT** protege contra borrados accidentales del padre.
- **ON UPDATE CASCADE** mantiene la consistencia si el ID del padre cambia.

*“LA HUMILDAD NO ES PENSAR MENOS
DE TI, ES PENSAR MENOS EN TI.
APRENDE DE TODOS, CRECE SIEMPRE”*

HUMILDAD

DUDAS